# Standard Library Tour Cheatsheet

**Lessons:** 19 — Standard Library Tour for Backend

**Examples:** —

**Covers:** `io`, `os`, `bufio`, `time`, `encoding/json`, `log/slog`, `flag`, `sort`, `regexp`

**Legend:** `[*]` = real Go API that the lesson has not covered yet

## io: the two interfaces everything speaks

```
io.Reader                   Read(p []byte) (n int, err error)
io.Writer                   Write(p []byte) (n int, err error)
io.Closer / io.ReadWriter / io.ReadCloser    the obvious combinations
io.ReadAll(r)               -> ([]byte, error) — careful with huge bodies
io.Copy(dst, src)           stream it; returns bytes copied
io.CopyN(dst, src, n)    [*] copy at most n bytes
io.LimitReader(r, n)     [*] a Reader that stops after n — the size cap
io.MultiReader(r1, r2)   [*] concatenate readers
io.MultiWriter(w1, w2)   [*] tee output to several writers
io.TeeReader(r, w)       [*] read from r, mirroring into w
io.EOF                      the sentinel that means "no more data"
io.Discard               [*] a Writer that throws everything away
io.NopCloser(r)          [*] give a Reader a no-op Close
(anything implementing these plugs into everything else — files, sockets, buffers)
```

## os: files, env, process

```
os.ReadFile(name)           -> ([]byte, error); one call, whole file
os.WriteFile(name, b, 0o644) create/truncate and write
os.Open(name)               -> (*os.File, error); read-only
os.Create(name)             -> (*os.File, error); create/truncate
os.OpenFile(n, flag, perm) [*] full control (O_APPEND, O_EXCL...)
defer f.Close()             always; check the error on writes
os.Remove / os.RemoveAll [*] delete a file / a tree
os.MkdirAll(path, 0o755) [*] mkdir -p
os.Stat(name)            [*] -> (FileInfo, error); os.IsNotExist(err)
os.Getenv("PORT")           "" when unset
os.LookupEnv("PORT")        -> (value, ok) — tells you unset from empty
os.Setenv / os.Environ   [*] write / list the environment
os.Args                     os.Args[0] is the program name
os.Exit(code)               immediate; NO defers run
os.Stdin / os.Stdout / os.Stderr    the three *os.File streams
os.UserHomeDir()         [*] cross-platform home directory
os.DirFS(dir)            [*] an fs.FS rooted at dir
```

## bufio

```
bufio.NewScanner(r)          line-by-line reading
  sc.Scan() / sc.Text() / sc.Bytes() / sc.Err()      the loop shape
  sc.Buffer(buf, max)    [*] raise the 64KB line limit
  sc.Split(bufio.ScanWords) [*] words, runes, or a custom splitter
bufio.NewReader(r)       [*] ReadString('\n'), ReadByte, Peek
bufio.NewWriter(w)           buffered writes — MUST Flush()
  defer w.Flush()            the line everyone forgets
bufio.NewReadWriter(r, w) [*] both halves at once
(scanner for lines, reader for control, writer to stop syscall-per-write)
```

## time

```
time.Now()                   the current wall + monotonic time
time.Since(start)            elapsed Duration — the timing idiom
time.Until(t)            [*] time remaining
time.Duration                an int64 count of NANOSECONDS
2 * time.Second              durations are typed constants you multiply
time.Millisecond / Second / Minute / Hour    the units
d.Seconds() / d.String() [*] convert / format a duration
time.Sleep(d)                block this goroutine
time.Parse(layout, s)        -> (Time, error)
t.Format(layout)             -> string
"2006-01-02 15:04:05"        THE reference layout (Mon Jan 2 15:04:05 MST 2006)
time.RFC3339                 the layout to use in APIs and logs
t.Unix() / time.Unix(s, ns)  seconds since the epoch
t.Add(d) / t.Sub(t2)         arithmetic
t.Before(t2) / After / Equal comparison — never use == on Time
time.Date(y, m, d, ...)  [*] construct an instant
t.UTC() / t.Local()      [*] change the location, not the instant
time.LoadLocation("UTC") [*] a *Location for a named zone
time.NewTicker / NewTimer / After     the channel-shaped APIs (concurrency sheet)
```

## encoding/json

```
json.Marshal(v)              -> ([]byte, error)
json.MarshalIndent(v, "", "  ")   pretty-printed
json.Unmarshal(b, &v)        pointer destination, always
json.NewEncoder(w).Encode(v) stream OUT to an io.Writer (adds a newline)
json.NewDecoder(r).Decode(&v)     stream IN from an io.Reader
dec.DisallowUnknownFields()  reject unexpected keys — do this on request bodies
dec.More()               [*] another value in the stream?
json.RawMessage          [*] keep a fragment undecoded
json.Valid(b)            [*] is it well-formed JSON?
`json:"name"`                rename
`json:"email,omitempty"`     omit when empty
`json:"-"`                   never encode this field
`json:",string"`         [*] encode a number as a string
MarshalJSON / UnmarshalJSON [*] custom encoding for a type
(only EXPORTED fields are marshalled; unexported ones vanish silently)
```

## log/slog (structured logging)

```
slog.Info("msg", "key", val, "key2", val2)    alternating key/value pairs
slog.Debug / Warn / Error    the four levels
slog.String("k", v) / slog.Int / slog.Bool / slog.Duration / slog.Any
                             typed attributes — faster and typo-proof
slog.New(slog.NewJSONHandler(os.Stdout, nil))     JSON for production
slog.New(slog.NewTextHandler(os.Stderr, nil))     key=value for humans
slog.SetDefault(logger)      make it the package-level default
logger.With("request_id", id)     a child logger carrying fixed fields
logger.InfoContext(ctx, ...) [*] pass the context through
&slog.HandlerOptions{Level: slog.LevelDebug, AddSource: true}   [*] options
slog.Group("user", ...)  [*] nest attributes
(one line per event, machine-parsable; never log secrets or full request bodies)
```

## flag

```
port := flag.Int("port", 8080, "listen port")     -> *int
name := flag.String("name", "", "usage")
debug := flag.Bool("debug", false, "usage")
d := flag.Duration("timeout", 5*time.Second, "")  [*]
flag.IntVar(&cfg.Port, "port", 8080, "")      [*] into an existing variable
flag.Parse()                 call it once, at the top of main
flag.Args() / flag.NArg()    the positional arguments left over
flag.Usage = func(){...} [*] custom help output
(env vars beat flags for containers — see the config sheet)
```

## sort & regexp

```
sort.Ints / sort.Strings / sort.Float64s     the pre-generics helpers
sort.Slice(s, func(i, j int) bool)           any slice, one closure
sort.SliceStable(s, less)    keeps equal elements in their order
sort.Search(n, f)            binary search over a predicate — the boundary tool
sort.Interface           [*] Len/Less/Swap, for custom collections
(prefer the generic slices package in new code — see the sorting sheet)

regexp.MustCompile(`^\d+$`)  compile once, at package level
re.MatchString(s)            -> bool
re.FindStringSubmatch(s) [*] -> the full match plus capture groups
re.ReplaceAllString(s, repl) [*] substitution
re.FindAllString(s, -1)  [*] every match
(Go's RE2 has no backtracking: linear time, and no ReDoS — but no backreferences)
```

## OTHER PACKAGES WORTH KNOWING [*]

```
context                     cancellation & deadlines (concurrency sheet)
net/http                    client and server (HTTP sheet)
database/sql                the DB pool (database sheet)
encoding/base64 / hex       binary <-> text
crypto/rand                 cryptographically secure random bytes
crypto/sha256 / hmac        hashing and signatures
path/filepath               Join, Base, Ext, Abs — OS-correct paths
net/url                     Parse, Values, QueryEscape
os/exec                     run external commands (arg SLICE, never a shell string)
errors                      Is / As / Join / Unwrap
math / math/big             numeric helpers
embed                       //go:embed files into the binary
```

## TRAPS & MEMORIZE

```
io.ReadAll on a request body  unbounded memory; wrap in http.MaxBytesReader
bufio.Writer without Flush    your output silently disappears
scanner's 64KB line limit     long lines stop the scan; sc.Err() tells you
os.Getenv can't see "unset"   use LookupEnv when empty is a valid value
Unmarshal into a non-pointer  silently does nothing useful
unexported fields in JSON     never marshalled, no error
time.Time == comparison       compares monotonic clocks too; use Equal
the layout string             it's the reference DATE, not yMd placeholders
Duration is nanoseconds       time.Duration(5) is 5ns, not 5s
os.Exit skips defers          and skips your Flush and Close calls
regexp compiled in a loop     compile once into a package var
logging a struct with %v      may print passwords; log explicit fields
```